

An Introduction to Programming through C++

Abhiram G. Ranade

Ch. 14: Arrays

Computers must deal with large amounts of data

- Simulate what happens when many balls are moving in a box. (Gas molecules?)
- Given altitudes of various points in a lake, find how much water is there given the water level.
- Given the road map of India, find the shortest route from Powai to Ganganagar.
- Analysing gravitational waves data to detect events like Black hole mergers
- For some problems, the input data is small, but for the computation we need to generate and store large amount of data.

How to handle lot of data?

- Fundamental problem: Writing out variable names to store information would be tiring

```
double pressure1, pressure2, ..., pressure1000;
```

- This is the problem solved using Arrays.
- More elaborate, modern, and flexible solution involving “vector”s will be discussed later.
- Arrays are simple to understand. Ideas useful in vectors too.

Outline

- How to create arrays, access elements
- Using arrays
 - Storing sequences
 - Storing sets
 - Storing “queues”
- Example programs

Arrays

```
double pressure[1000];
```

- Essentially defines 1000 variables (“array elements”).
- Variables are named `pressure[0]`, `pressure[1]`, `pressure[2]`, ..., `pressure[999]`
- General form:

```
data-type array-name[size];
```

- size also called length.
- `array-name[index]` gives index^{th} variable.
- Necessary: $0 \leq \text{index} < \text{size}$.
- **Not just a collection of variables:** Index may be given as an expression.

Array element operations

```
double pressure[1000];  
cin >> pressure[0] >> pressure[2];  
pressure[1] = (pressure[0] + pressure[2])/2;  
for(int i=0; i<1000; i++) cin >> pressure[i];  
cout << pressure[439]*3.33 << endl;
```

```
for(int i=1; i<999; i++)  
    pressure[i] = (pressure[i-1] + pressure[i+1])/2;
```

- Array index can be an expression which will be evaluated during execution and then the corresponding element will be used.
- Not possible with 1000 names.

Index out of range

```
double pressure[1000];  
pressure[1000] = 1.2;  
double d = pressure[-5];
```

- In the assignments above, the array index is outside the allowed range: **0 through size-1**.
- If the array index is out of range, the program may run and produce wrong results, may halt with a message. Nothing is guaranteed.
- **The programmer must ensure index stays in range.**

Initialization while defining

```
int squares[4] = {0, 1, 4, 9};  
int cubes[] = {0, 1, 8, 27, 64};  
// size = 5 inferred.
```

```
int x, pqr[200], y[]={1,2,3,4};
```


Exercises

- Define an array to store 10 telephone numbers
 - Assume telephone numbers are 9 digit long
 - Assume telephone numbers are 12 digit long
- Define an array to store the first 5 primes, and initialize it to be the 5 primes.

What we discussed

An array is a collection of variables of the same type.

$A[0], A[1], \dots, A[n-1]$

Where A is the array name, [i] is the index.

Key point: the index can be specified by an expression which is evaluated and the result used to determine which variable is meant.

- How to declare arrays, with and without initialization.
- How to get values into and out of array elements.
- Next: Examples of use



Attendance display problem

- Read in attendance percent of the 345 students in a class, with in roll number order, 0 to 344.
- After that:
 - Students may arrive in any order, and give their roll number.
 - The program must respond by printing out their marks.
 - If any illegal number is given as roll number, the program must terminate.

Program

```
double attend[100];

for(int i=0; i<100; i++)
    cin >> attend[i];

while(true){
    int rollno;
    cout << "Roll no:";
    cin >> rollno;
    if(rollno < 0 || rollno > 99) break;
    cout << attend[rollno] << endl;
}
```

Display who has highest attendance

```
// marks defined and read into as before.
double maxsofar = attend[0];
for(int i=1; i < 345; i++){
    // Plan: at the beginning of the ith iteration,
    // maxsofar should hold the maximum of marks[0..i-1].
    maxsofar = max(maxsofar, attend[i]);
}
// maxsofar now holds max value in attend[0..344].

for(int i=0; i < 345; i++)
    if(attend[i] == maxsofar) cout << i << endl;
```

- Accumulating
- Going over the array to filter elements that match a certain condition: also standard.

Exercise

Usually we will have roll numbers starting at 1 rather than at 0. To handle this, a simple strategy is to store the marks of roll number i in `marks[i-1]`.

Modify both the programs discussed in this segment to follow this convention.

What we discussed

- Some simple programs involving arrays.
- Point to note: we did not store the roll numbers, because the index played the role of the roll numbers.
- Programs involved idioms such as
 - Going through all elements of the array and filtering out elements to print.
 - Accumulating the maximum in a variable. Other kinds of accumulation is also possible, e.g. sum.
- Next: more examples



Histogram

Read in attendance as before, print how many have between 0-9, 10-19, 20-29, ..., 90-99, and exactly 100.

```
int hist[11];  
// Plan: hist[i] will store number of  
// students having attendance between  
//  $10*i$  and  $10*(i+1)-1$ 
```

- On reading a certain attendance percent v , add 1 to suitable element of hist.
- Which element?

Histogram

```
for(int i=0; i<11; i++)  
    hist[i]=0;  
for(int i=0; i<345; i++){  
    double attend; cin >> attend;  
    hist[ int(attend)/10 ]++;  
    // int(..) converts to int.  
}
```

What we discussed

- Histogram computation
- Key ideas:
 - Index is calculated in an unusual manner
 - Integer division plays an important role

Next: attendance display variation



Attendance display variation

- Roll numbers are not in range 1 .. 100, but a larger range, e.g. 1352.
- Attendance list = 345 pairs of numbers: (rollno, attendance),
- Teacher must enter roll number, marks into the computer.
- Later:
 - Students arrive and each student types in roll number r .
 - Program must print out attendance if r is valid roll number.
 - If r is -1, then stop.

Program idea:

- Use an additional array for storing roll numbers.
- Store i th roll number in `rollno[i]` and i th attendance into `attend[i]`.
- When students arrive: Examine each element of the roll number array and see if it equals r . If so print corresponding attendance.

The program

```
int rollno[345]; double attend[345];
for(int i=0; i<345; i++)  cin >> rollno[i] >> attend[i];

while(true){
    int r; cin >> r;        // read in query roll number
    if(r == -1) break;
    bool found = false;
    for(int i=0; i<345; i++){
        if(rollno[i] == r){
            cout << attend[i] << endl;
            found = true;
            break;
        }
    }
    if(!found) cout << "Roll number not found.\n";
}
```

Exercise

- Modify the program so that there are attendance for lectures and labs.
- Write a program that takes attendance data and prints the number of students who are within the range $[\mu - \sigma, \mu + \sigma]$

Here μ is the average and σ is the standard deviation

What we discussed

- If the roll numbers are not in the range $0..N-1$ where N is the number of students, then we need to store the roll numbers also.
- Searching through the array to find data corresponding to a “key” is a common idiom.
 - Use of a “found” variable is a natural strategy

Next: Polynomial multiplication



Polynomial multiplication

- Given polynomials $A(x)$, $B(x)$

$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

$$B(x) = b_0 + b_1x + b_2x^2 + \dots + b_mx^m$$

- Find their product $C(x) = A(x) B(x)$

$$C(x) = c_0 + c_1x + c_2x^2 + \dots + c_{m+n}x^{m+n}$$

- Given $a_0, \dots, a_n$ and $b_0, \dots, b_m$ find $c_0, \dots, c_{m+n}$
 - Natural to use an array of $n+1$ elements to store the coefficients of a degree n polynomial.
- Algorithm idea:
 - Each term a_ix^i in $A(x)$ will multiply each term b_jx^j in $B(x)$ and the product $a_ib_jx^{i+j}$ will contribute to the term $c_{i+j}x^{i+j}$.

Program to multiply degree 10 polynomials

```
double a[11], b[11], c[21];
// a, b have degree 10, c has degree 20.
for(int i=0; i<=10; i++)
    cin >> a[i];                // read in polynomial A(x)
for(int j=0; j<=10; j++)
    cin >> b[j];                // read in polynomial B(x)
for(int k=0; k<=20; k++)
    c[k] = 0;
for(int i=0; i<=10; i++)
    for(int j=0; j<=10; j++)
        c[i+j] += a[i]*b[j];    // as discussed earlier.
for(int k=0; k<=20; k++)
    cout << c[k] << ' '; // output c, separated by spaces
cout << endl;
```


Polynomial multiplication: Another algorithm

- Given polynomials $A(x)$, $B(x)$

$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

$$B(x) = b_0 + b_1x + b_2x^2 + \dots + b_mx^m$$

- Find their product $C(x) = A(x) B(x)$

$$C(x) = c_0 + c_1x + c_2x^2 + \dots + c_{m+n}x^{m+n}$$

$$c_k = a_0b_k + a_1b_{k-1} + \dots + a_jb_{k-j} \dots + a_kb_0$$

- We can calculate c_k directly:

$$c[k] = 0;$$

$$\text{for}(\text{int } i = \underline{\hspace{1cm}} ; i \leq \underline{\hspace{1cm}} ; i++) \quad c[k] += a[i] * b[k-i];$$

Exercise

- Complete the program to multiply polynomials using the second method.
- Write the program for multiplying a degree m polynomial by a degree n polynomial.
- Estimate how many multiplications are performed in multiplying a degree m polynomial by a degree n polynomial.

What we discussed

- Polynomials can be nicely represented by arrays.
 - Coefficient of the i th power can be stored in i th element
 - We do not need to store i explicitly.
 - Similar to the case when we had consecutive roll numbers starting at 0.
- Multiplication can be done in different ways.
 - Be sure you understand the the array indices for the second method.

Next: Taxi dispatch



Dispatching Taxis

- Taxi drivers arrive: driverID put into “queue”.
 - driverID : integer
- Customer arrives:
 - If taxi is waiting, first driver in queue is assigned.
 - If no taxi waiting, customer asked to check again later.

Key requirements

- Remember driverIDs of drivers who are waiting to pick up customers.
- Remember the order of arrival.
- When customer arrives:
 - assign the earliest driver.
 - Remove driverID of assigned driver from memory.
- When driver arrives: Add driver's driverID to memory.

How to remember driverIDs

- Use an array.

```
const int n=500;  
int driverID[n];
```

- n: maximum number of drivers that might have to wait simultaneously.
- In what **order** to store the ids in the array?
- What **other information** do we need to remember?
- What do we do when customer arrives?
- What do we do when driver arrives?

Idea 1

- Store earliest driver in `driverID[0]`.
- Next earliest in `driverID[1]`. ...
- Remember number of drivers waiting.

```
int nWaiting;
```

Program outline

```
const int n=500; int driverID[n], nWaiting = 0;
while(true){
    char command; cin >> command;
    if(command == 'd'){
        // process driver arrival.
    }
    else if(command == 'c'){
        // process customer.
    }
    else if(command == 'x') break; // to terminate program
    else cout << "Illegal command.\n";
}
```


Invariants/Plan

- `nWaiting` = number of waiting drivers.
- Number of waiting drivers can be at most the array length
 $0 \leq nWaiting \leq n$
- id of earliest waiting driver is in `driverID[0]`.
- Next in `driverID[1]`,
- ...
- Last in `driverID[nWaiting-1]`.

Driver arrival

```
if(nWaiting == n)
    cout << "Queue full.\n";
else{
    int d; cin >> d;
    driverID[nWaiting] = d;
    nWaiting ++;
}
//Do invariants hold?
```

When customer arrives:

Provided $nWaiting > 0$:

- Assign the earliest unassigned driver to customer.
 - Earliest unassigned stored in `driverID[0]`.
- Second earliest should become new earliest...
- Third earliest should become ...
- `nWaiting` should decrease.

Customer Arrival

```
if(nWaiting == 0)
    cout << "Try again later.\n";
else{
    cout << "Assigning " << driverID[0]
        << endl;
    for(int i=1; i <= nWaiting - 1; i++)
        driverID[i-1] = driverID[i];
    // Queue shifts up
    nWaiting-- ;
}
```

What we discussed

- A workable scheme for matching taxis to customers.
- Taxi drivers are recorded in an array in the order of their arrival.
 - Arrival order not explicitly stored
 - Implicitly encoded in the index of the element, or the order in which driverIDs are stored in memory.

Next: Improvement to this scheme

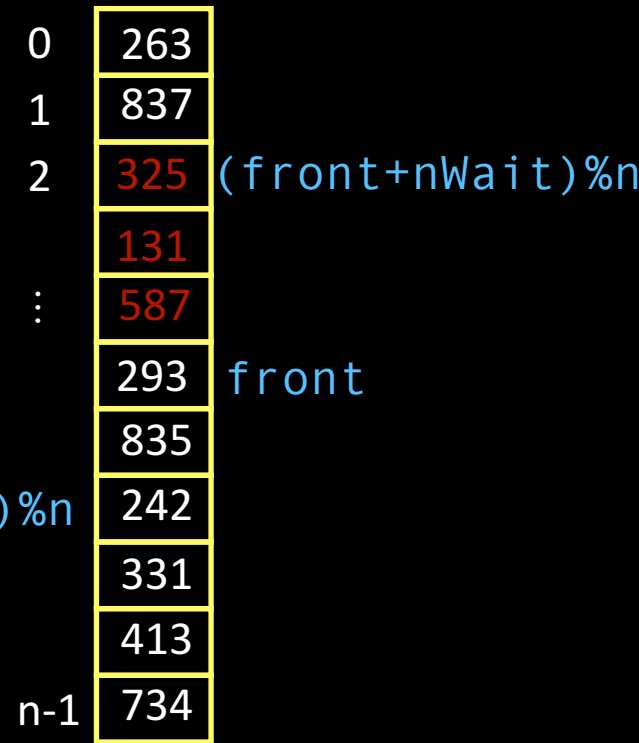
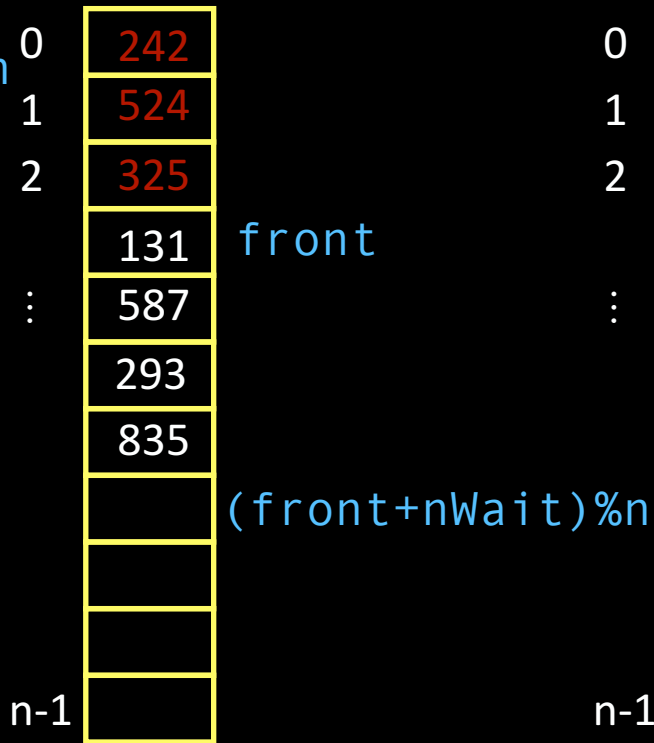
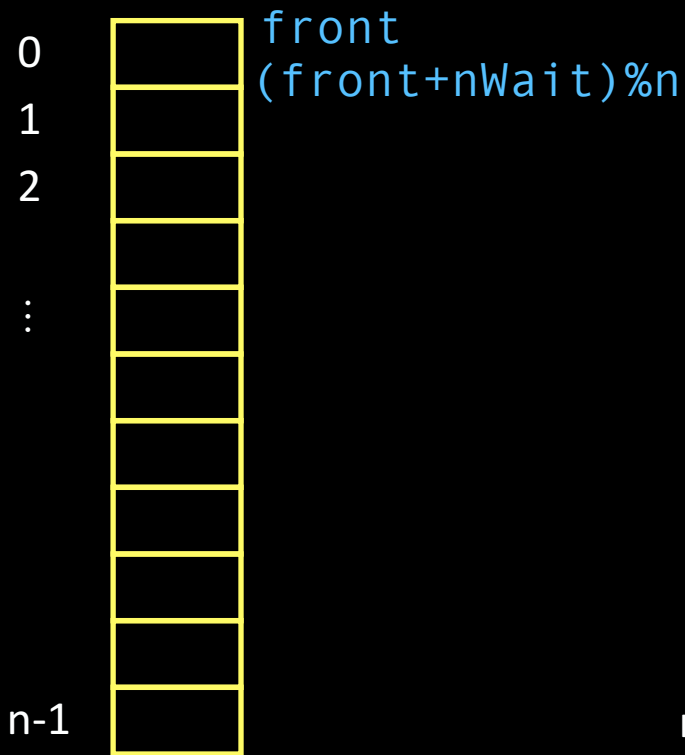


Taxi Dispatch: Idea 2

- Previous program seems to be copying data a lot...
 - Can we avoid the copying?
- Emulate what might happen without computers.
 - Names written on blackboard.
 - Arriving driver IDs written top to bottom.
 - When board bottom reached, begin from top if drivers have left.
 - Think of driver ID as a circular array.
 - “Next” position after driver ID[n-1] (bottom of board) is driver ID[0] (top of board).

Invariants

- $nWaiting$ = number of waiting drivers. $0 \leq nWaiting \leq n$
- New variable $front$: position of earliest unassigned driver
= 0 initially.
 $0 \leq front < n$
- First valid driver ID is at $driverID[front]$
- Where is the next? $driverID[front+1]$?
 - If $front = n-1$, then $front+1 = n$, which is not a valid index.
 - Next waiting driver is $driverID[(front+1) \% n]$
 - Last waiting driver id is: $driverID[(front+nWaiting-1) \% n]$
- Note that $\%$ provides the effect of “wrapping around”.



Note: Ids in red color are no longer in the queue

Processing driver arrival

```
if(nWaiting == n)
    cout << "Queue full.\n";
else{
    int d; cin >> d;
    driverID[(front+nWaiting) % n] = d;
    nWaiting ++;
}
```

```
// front + nWaiting % n : index of
// empty position after end of queue.
```

Processing Customer Arrival

```
if(nWaiting == 0)
    cout << "Try later.\n";
else{
    cout << "Assigning " <<
        driverID[front] << endl;
    front = (front + 1) % n;
    nWaiting --;
}
```

What we have discussed

- Different idea for solving taxi dispatch problem.
- New idea is better, copying of elements of `driverID` is avoided.
- Exercise: verify that the invariants indeed remain true after each customer or driver arrival.

Next: Disk intersection problem



Disk intersection problem

- Given a collection of n disks in the plane, decide if any two intersect.

Algorithm: check all possible pairs.

- Let i = index of smaller numbered disk.
- So i must range from 0 to $n-2$
- For each i , the other disk must range from $i+1$ to $n-1$.

Intersection check:

- Distance between centers must be less than sum of the radii.

What we have discussed

- Given a set of objects, how do you go over all pairs.
- Exercise: Can you generalize the idea?
 - Write a program to check if any 3 of a given set of n points in the plane are collinear.



Arrays: concluding remarks

- Way to store many objects of the same type in memory, without having to separately define a variable for each.
- Index: used to choose an element.
 - Must be at least zero and at most array length $- 1$.
 - Can be an expression
- Index may sometimes play an active role:
 - When roll numbers are consecutive from 0 to $N-1$
 - Polynomial representation: i th coefficient stored in i th element
 - Taxi dispatch: indicates arrival order
- Index may not have significance:
 - When roll numbers took on arbitrary values
- Indexing into an array happens very fast, independent of how many elements are present in the array.

More remarks

Several idioms used with arrays:

- Scan through all elements to find a matching element (“linear search”)
- Do some calculation which determines the index where something is to be done (“histogram”)
- Use the array like a queue of elements which wait for something to happen, after which they leave the queue (“taxi dispatch”)



Pointers

- If the memory of a computer has N bytes, then the bytes are numbered $0..N-1$.
- The number of a byte (different from what is stored in the byte) is said to be its **address**.
- A **pointer** is a variable that can store addresses.
 - Sometimes “pointer” might mean address.
- **What we accomplished using pass by reference can also be accomplished using pointers.**
 - This will be seen soon.
- Pointers will also be useful elsewhere.

How to find the address of a variable

- The operator **&** can be used to get the address of a variable.
 - The same & is used to mark reference parameters; but the meaning will be clear from the context.

```
int t;  
cout << &t << endl;
```

- This prints the address of the variable t, i.e. the address of the first byte that comprises the variable t.
- Customarily, addresses get printed in hexadecimal radix, i.e. they will consist of a sequence of hexadecimal digits prefixed by “0x”
- Note: hexadecimal digits: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F.

Variables that can store addresses

- How to create a variable for storing **addresses of variables of type `int`** :

```
int *v;    // read as “int star v”
```

- The `*` is not to be read as multiplication.
 - Think of it as `(int*) v`; where `int*` means the type: “address of `int`”.

```
int p;
```

```
v = &p;    // address of p stored in v
```

- Since `p` is of type `int`, `&p` has type address of `int*`.
- Thus, it is OK to store `&p` in `v`, which is also of type address of `int`.

```
cout << v << ' ' << &p << endl; // both print same
```

```
v = p; // compile time error: type mismatch
```

Pointers in general

- In general, to create a variable w to store addresses of variables of type T , write:

$T^* \ w;$

- Assignment statements: types of lhs and rhs must be same
 - Except when both sides are numeric types; then conversion rules used.
 - No conversion rule between pointers of one type and pointers of other types.
 - No conversion rule between pointers of one type and values of any type.

The dereferencing operator *

- If `v` contains the address of `p`, then we can get to `p` by writing `*v`.

```
int* v;
```

```
int p;
```

```
v = &p;
```

```
*v = 10; // as good as p = 10.
```

- Think of `*` as the inverse of `&`.
- `&p` : the address of the variable `p`
- `*v` : the variable whose address is in `v`
- `int *v`;
 - `v` is such that `*v` is an `int`
 - `v` is an address of an `int`

Pointers in functions

```
void CtoP(double x, double y,  
    double* pr, double* ptheta){  
    *pr = sqrt(x*x + y*y);  
    *ptheta = atan2(x,y);  
    return;  
}  
main_program{  
    double r, theta;  
    CtoP(1,1,&r,&theta);  
    cout << r << ' '  
        << theta << endl;  
}
```

main_program calls CtoP, supplying `&r`, `&theta` as third and fourth arguments.

This is acceptable because **corresponding parameters** have type `double*`.

- The first step of the call copies the addresses of `r`, `theta` of the `main_program` into `pr`, `ptheta` of `CtoP`.
- `*pr` means the variable whose address is in `pr`, in other words, the variable `r` of `main_program`.
- Thus `CtoP` changes the variables of `main_program`.
- Thus `r` becomes $\sqrt{2} = 1.41$ and `theta` becomes $\pi/4 = 0.79$ and are printed.

Remarks

- In variable definitions, * associates to the right. Example:

```
int *v, p;
```

- This means `int *v;` and `int p;` i.e. defines a variable `v` of type `int*`, and variable `p` of type `int`.
- For now, assume that the only operations you can perform on a variable of type `T*` are
 - dereference it,
 - store into it a value `&v` where `v` is of type `T`,
 - store it into another variable of type `T*`
 - pass it to a function as an argument, provided corresponding parameter is of type `T*`

Exercise

- Point out the errors in this code.

```
int *p, *q, w;  
p = w;  
q = 3;
```

- What is the result of executing the following:

```
int *p, *q, w, x;  
p = &w;  
w = 10;  
q = &x;  
*q = 20;  
cout << *p + x << endl;
```

What we discussed

- A pointer is an address or a variable containing an address.
- A pointer to a variable can be created in the main program and dereferenced during a function call.
- This way a function can be allowed to modify variables in the main program, or other functions.
- Pointers can do the same thing as references, but the notation is clumsier.
- But pointers can do other things too. (Later)



Passing arrays to a function

```
int average(int a[], int n){
    float sum=0;
    for(int i=0; i<n; i++)
        sum = sum+a[i];
    return sum/n;
}
int main(){
    int b[4] = {0, 4, 2, 6};
    cout << average(b, 4) <<
    endl; // prints 3
    cout << average(b, 2) <<
    endl; // prints 2
}
```

When array is passed as an argument, its address is passed.

If you change `a[i]`, it will affect the array in the main function as well.

If you need the length of the array in the function, it should be passed as a parameter